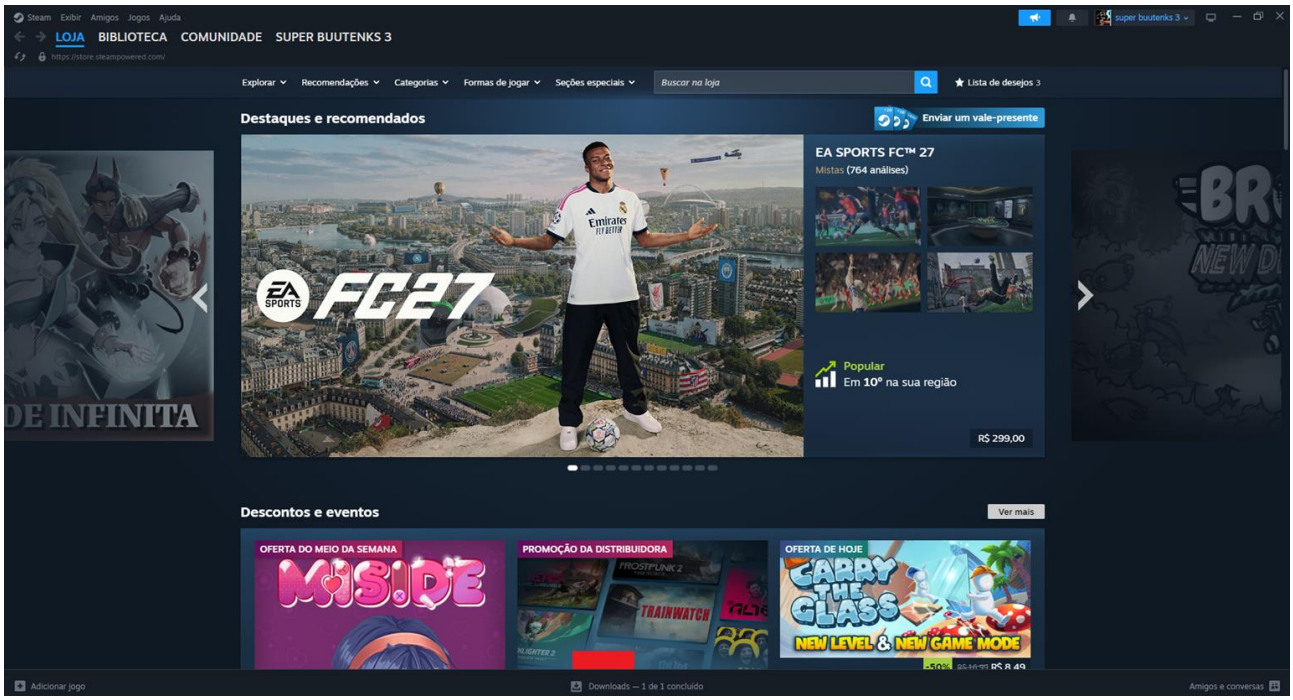
INF 01145
FUNDAMENTOS DE BANCOS DE DADOS
TRABALHO FINAL

Modelando a Steam usando o Modelo Relacional

por Lucas Hailliot Dahmer e Davi Lopes Hoehene
Prof.ª Karin Becker



A Steam é uma plataforma de venda de jogos eletrônicos. Serve não apenas como o lugar para comprar, mas também para jogar a maioria dos jogos que vende, usando seu sistema de contas para ligar usuários aos serviços multiplayer dos jogos.

As funções da plataforma que decidimos implementar foram as seguintes:

- Criação de conta/login;
- Amizades entre usuários;
- Loja de jogos filtrável;
- Biblioteca de jogos do usuário;
- Avaliação de jogos por usuário.

Além dos citados acima, a plataforma também oferece diversos outros recursos, como a Oficina da Comunidade, onde os usuários podem criar conteúdo customizado para determinados jogos, e o *Marketplace*, onde usuários podem trocar itens de certos jogos entre si, além de vendê-los em um mercado autorregulado, onde quem determina os preços são os próprios anunciantes. Decidimos não implementar a Oficina e o Marketplace, visando manter o escopo do projeto mais bem definido e evitar que as features a serem implementadas saiam de controle.

Abaixo estão listadas as funcionalidades planejadas do sistema e seus requisitos funcionais:

Login/criação de conta:

Para isso, usamos a tabela conta, que vai armazenar a maioria dos dados relativos a uma única conta incluindo senha e e-mail. O e-mail de uma conta é único, com id_conta sendo chave primária e e-mail sendo uma chave alternativa.

Compra de itens via loja:

A compra de um jogo deve ser possível se um usuário tiver saldo suficiente. Para isso, haverá uma funcionalidade de depósito de saldo. Para isso, utilizaremos as tabelas jogo, pedido, e item_pedido. A tabela pedido representa uma compra de jogos feito por uma conta, e os jogos desse pedido são listados na tabela item_pedido, que associa um jogo a um pedido.

Filtragem da loja por preço/gênero/desenvolvedora:

A loja poderá ser filtrada de diferentes formas: simples para preço/desenvolvedora pois são atributos de um jogo (desenvolvedora é uma tabela própria referenciada na tabela jogo); para filtrar por gênero, usamos a tabela jogo_gênero, que associa um gênero (instanciado em uma tabela própria) a um jogo, permitindo que um jogo tenha mais de um gênero listado.

Sistema de amizade:

O sistema de amizades entre contas utiliza a tabela amizade, que associa duas contas: um solicitante e um destinatário. Isso porque a tabela modelará um pedido de amizade, podendo assumir os estados ‘pendente’, ‘aceita’ ou ‘recusada’. Para mostrar a lista de amigos da conta c1, podemos selecionar da tabela amizade os elementos que possuam c1 como qualquer uma das contas e tenha o estado ‘aceita’.

Sistema de avaliação de jogos:

Na steam, um usuário pode deixar uma avaliação em um jogo em forma de uma recomendação, sendo ela positiva ou negativa. Em nossa versão, a loja constará com avaliações de usuários também, podendo deixar uma mensagem e uma nota de 1 a 5 (definidos como inteiros em uma lista, não como intervalo real) aos jogos. Para isso, usaremos a tabela avaliacao, que associa as tabelas conta e jogo. Pensamos em implementar a filtragem da loja por pontuação baseada em avaliações, também.

Sistema de biblioteca:

A tabela biblioteca também associa as tabelas conta e jogo, mas consta com atributos diferentes, como data de aquisição e tempo de jogo. Poderá ser visualizada pelo usuário fazendo uma consulta à tabela usando id_conta como filtro.

Esquema Relacional

Segue abaixo o esquema relacional e suas DFs, feito com assistência de IA, derivado do arquivo contendo as tabelas SQL na FNBC:

Conta (id_conta, username, email, senha, data_nascimento, pais, saldo)
- Chave primária: id_conta
- Únicos: e-mail, username
  - id_conta → username, e-mail, senha, data_nascimento, pais, saldo
  - e-mail → id_conta, username, e-mail, senha, data_nascimento, pais, saldo

username → id_conta, e-mail, senha, data_nascimento, pais, saldo

Amizade (id_solicitante, id_destinatario, status)
Chave primária: (id_solicitante, id_destinatario)
id_solicitante referencia Conta (id_conta)
id_destinatario referencia Conta (id_conta)
id_solicitante, id_destinatario → status

Desenvolvedora (id_desenvolvedora, nome, pais)
Chave primária: id_desenvolvedora
Únicos: nome
id_desenvolvedora → nome, pais
nome → id_desenvolvedora, pais

Genero (id_genero, nome)
Chave primária: id_genero
Únicos: nome
id_genero → nome
nome → id_genero

Jogo (id_jogo, id_desenvolvedora, titulo, preco)
Chave primária: id_jogo
Únicos: titulo
id_desenvolvedora referencia Desenvolvedora (id_desenvolvedora)
id_jogo referencia Jogo (id_jogo)
id_jogo → id_desenvolvedora, titulo, preco
titulo → id_jogo, id_desenvolvedora, preco

Jogo_Genero (id_jogo, id_genero)
Chave primária: (id_jogo, id_genero)
id_jogo referencia Jogo (id_jogo)
id_genero referencia Genero (id_genero)
Nenhuma DF não trivial

Pedido (id_pedido, id_conta, data_pedido)
Chave primária: id_pedido
id_conta referencia Conta (id_conta)
id_pedido → id_conta, data_pedido

Item_Pedido (id_pedido, id_jogo, preco_pago)
Chave primária: (id_pedido, id_jogo)
id_pedido referencia Pedido (id_pedido)
id_jogo referencia Jogo (id_jogo)
id_pedido, id_conta → id_jogo

Biblioteca (id_conta, id_jogo, horas_jogadas, ultima_sessao)
Chave primária: (id_conta, id_jogo)
id_conta referencia Conta (id_conta)
id_jogo referencia Jogo (id_jogo)
id_conta, id_jogo → data_compra, horas_jogadas, ultima_sessao

Avaliacao (id_jogo, id_conta, nota, comentario, data_avaliacao)
Chave primária: (id_jogo, id_conta)
(id_jogo, id_conta) referencia Biblioteca(id_jogo, id_conta)
id_jogo, id_conta → nota, comentário, data_avaliacao